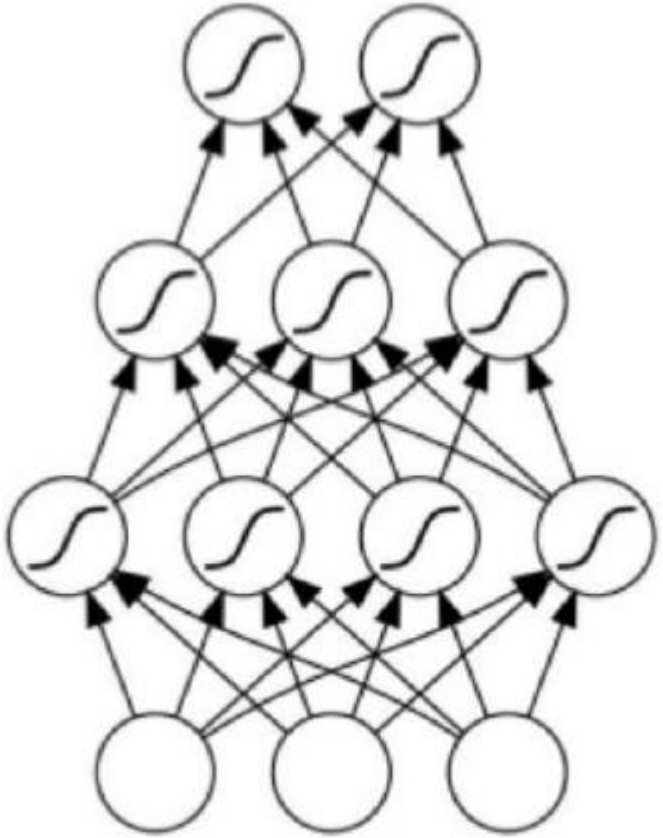
Технологии нейросетевых вычислений

Лекция №4. Рекуррентные сети

Запорожцев Иван Федорович
zaporozhtsev.if.work@gmail.com

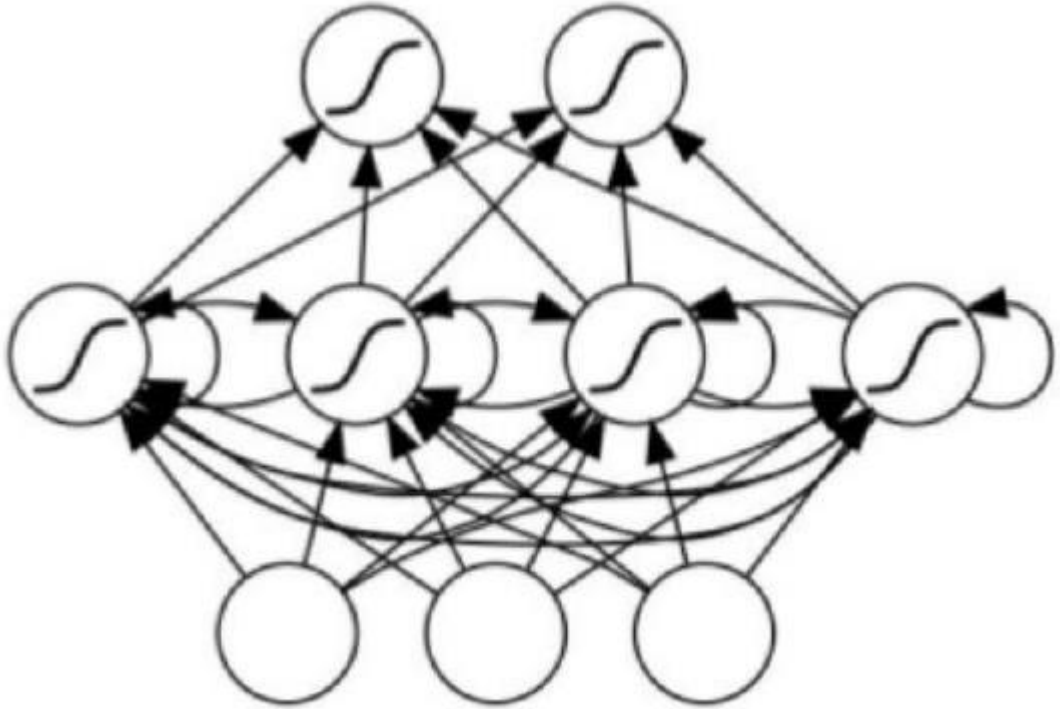
Модификация Feed Forward Network

Рекуррентная нейросеть (RNN = Recurrent Neural Network)



универсальный аппроксиматор

Идея: в сеть прямого распространения добавить рекуррентные связи



полнота по Тьюрингу

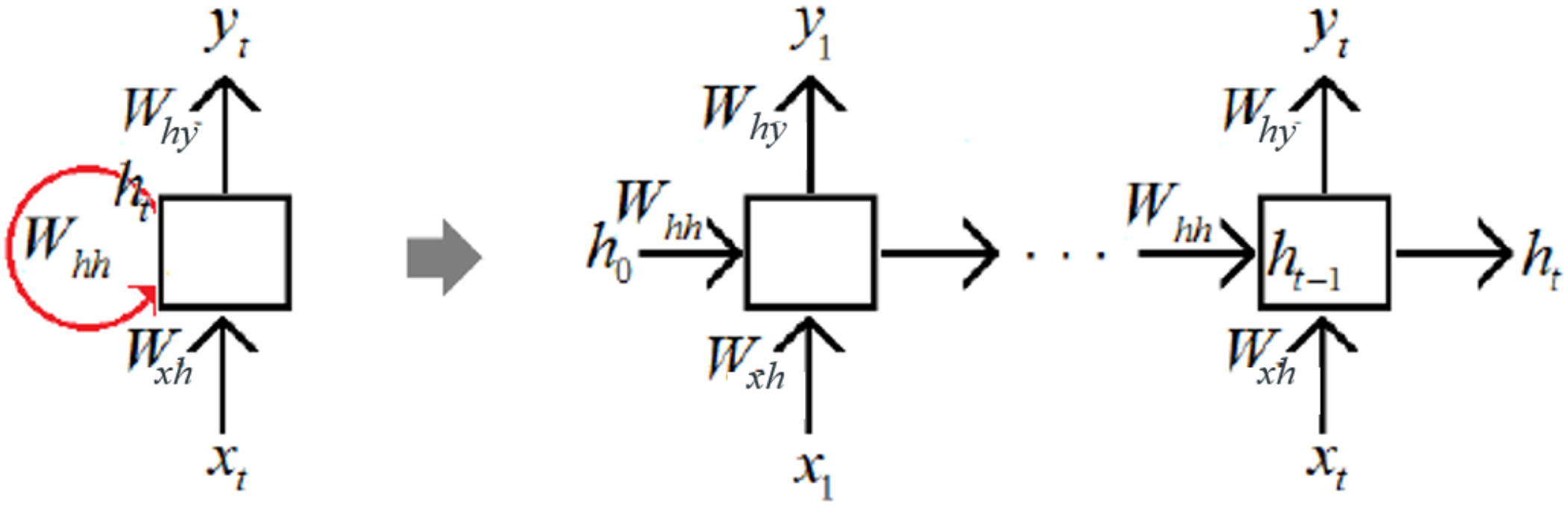
Модификация Feed Forward Network

Рекуррентная нейросеть (RNN = Recurrent Neural Network)

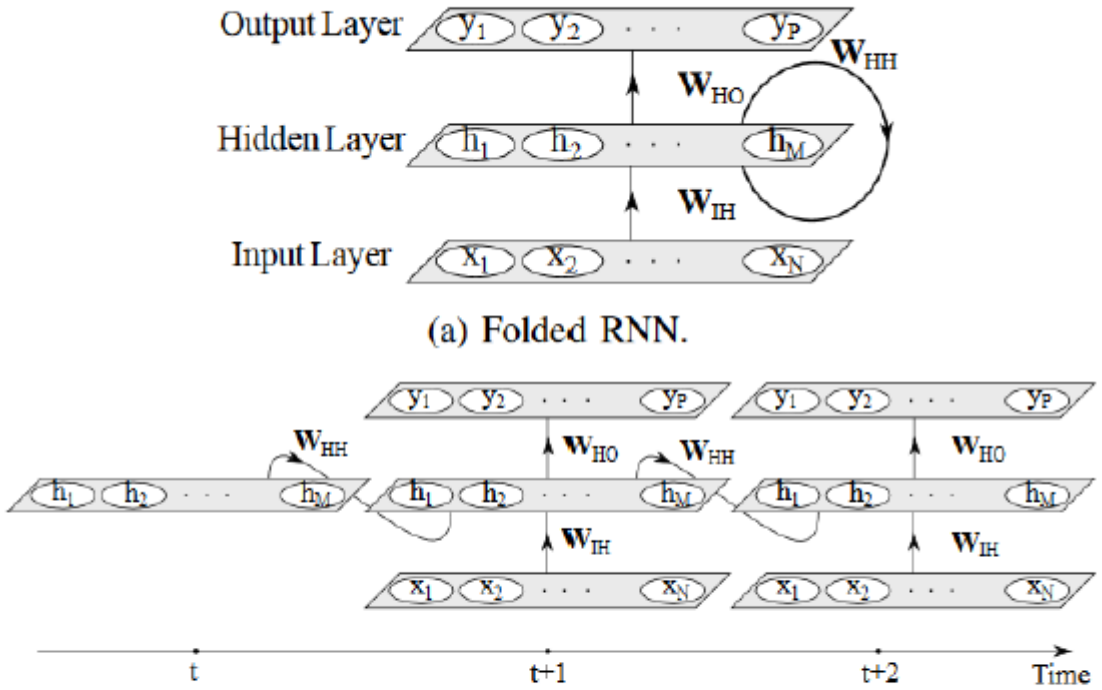
– для обработки / генерации последовательностей

использование выхода (output) / скрытого состояния (hidden state)

легко масштабируется при увеличении длины последовательностей



RNN: основная идея



(a) Folded RNN.

(b) Unfolded RNN through time.

Fig. 1: A simple recurrent neural network (RNN) and its unfolded structure through time t. Each arrow shows a full connection of units between the layers. To keep the figure simple, biases are not shown.

Главная идея – разделение параметров (parameter sharing)

как и в свёртках;)

Матрицы весов одинаковые при обработке любого элемента последовательности (символ, слово, ...)

Учим одну модель, которая применяется на каждом шаге к последовательности любой длины

Вариант реализации сети с одним рекуррентным слоем

$$\dim x = N,\ \dim h = M, \dim y = P$$

$$h_0 = init()$$

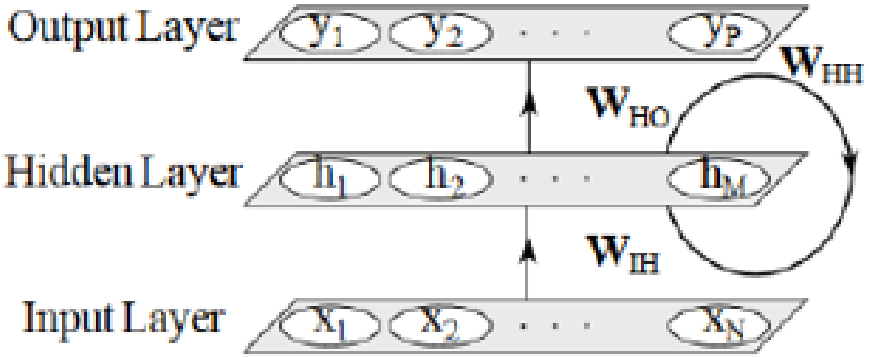
$$h_1 = \sigma\left(W_{hh} h_0 + W_{xh} x_1 + b_h\right) - вектор - функция$$

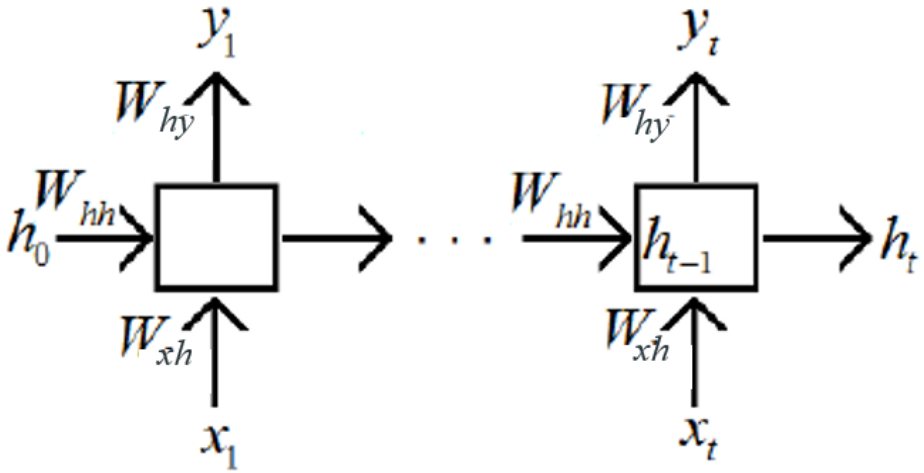
...

$$h_t = \sigma\left(W_{hh} h_{t-1} + W_{xh} x_t + b_h\right)$$

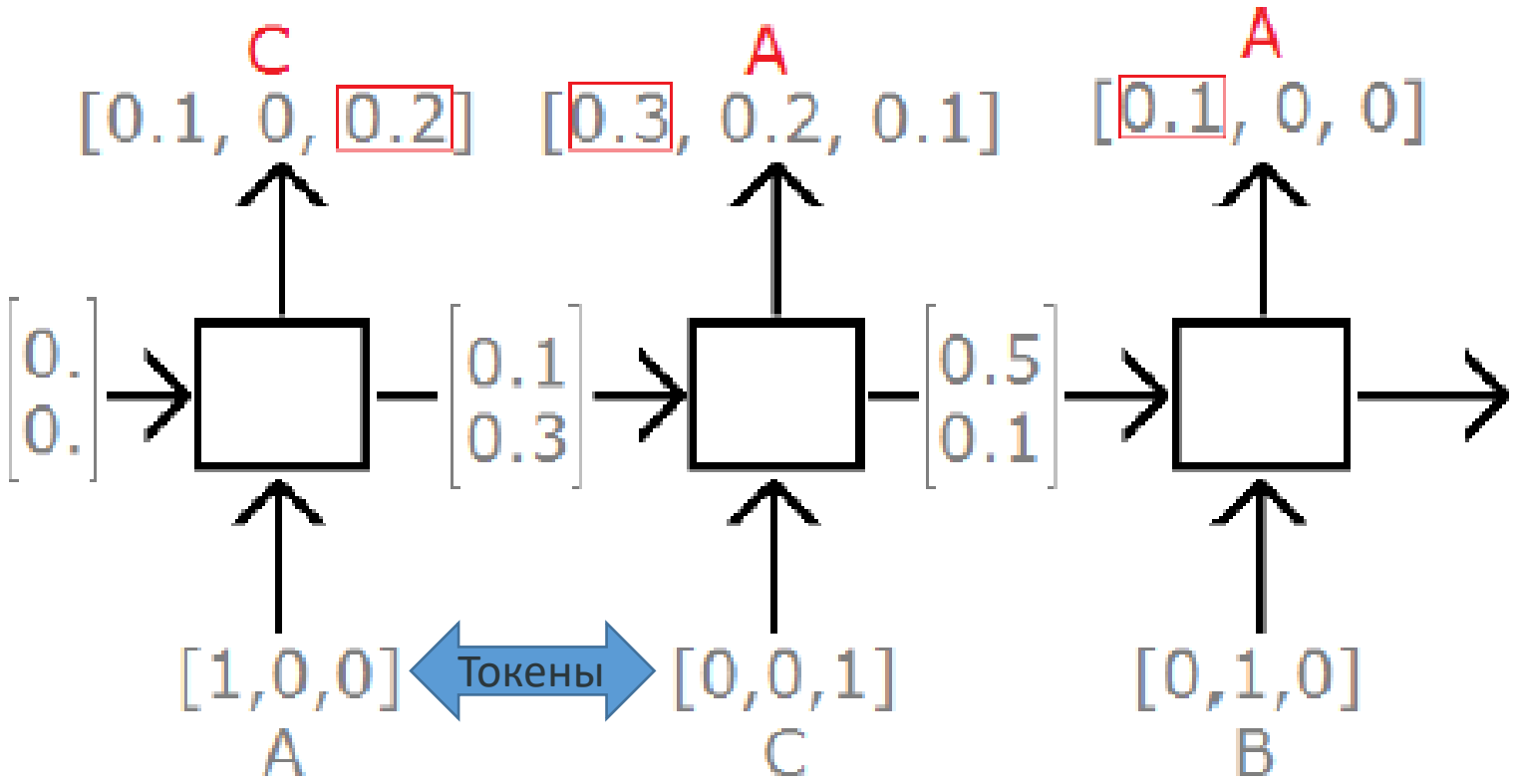
$$y_t = g\left(W_{hy} h_t + b_y\right) - вектор - функция$$

$$h_t = \sigma\left(\left[W_{hh}, W_{xh}\right]\begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_h\right) = \sigma\left(W\left[h_{t-1}, x_t\right]^T + b_h\right)$$



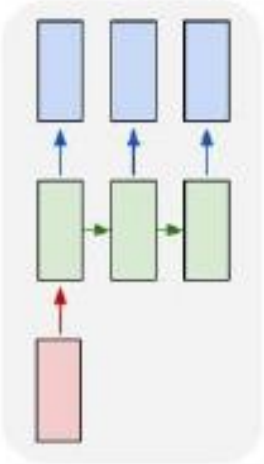


Обработка символьных последовательностей



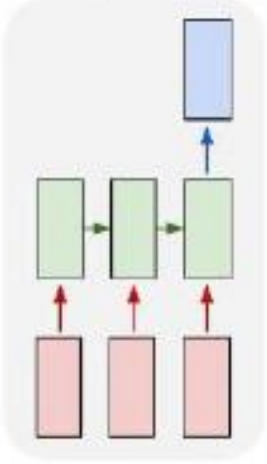
Варианты применения

one to many



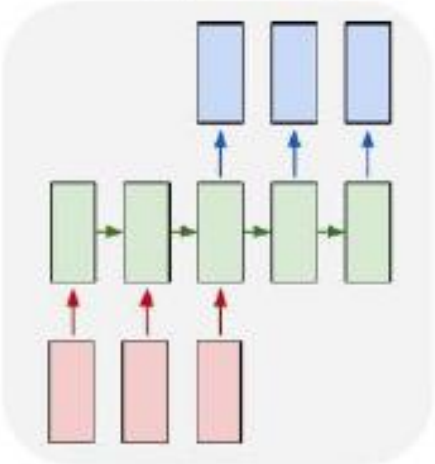
описание изображения

many to one



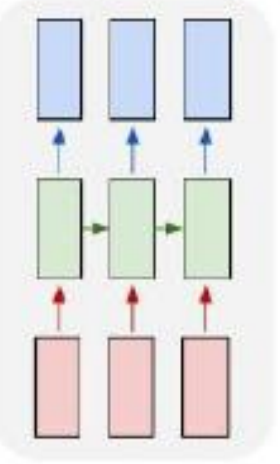
тема / настроение текста

many to many



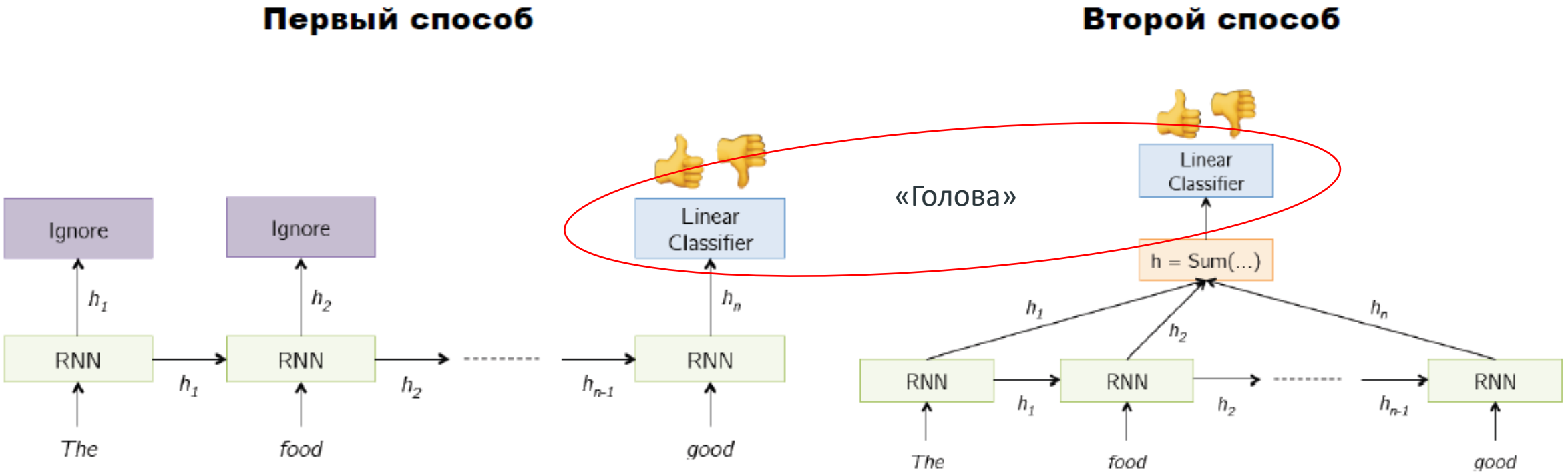
машинный перевод

many to many

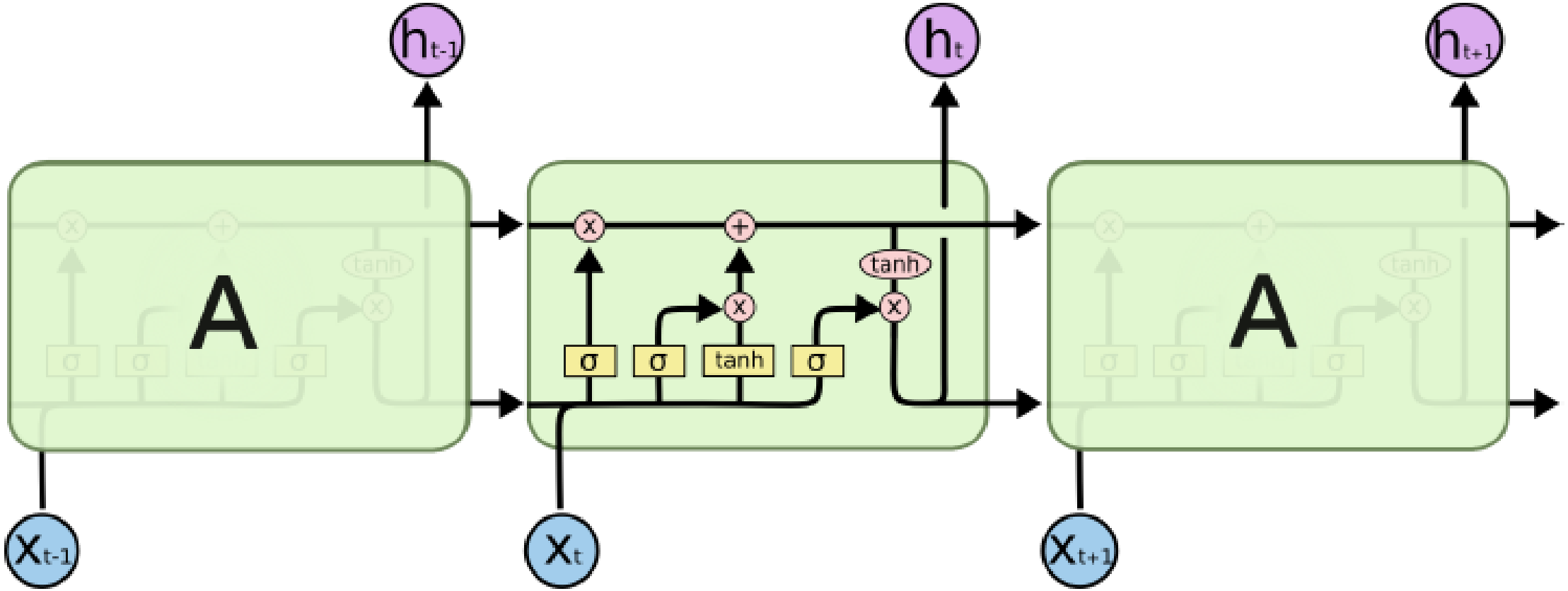


классификация фреймов видео

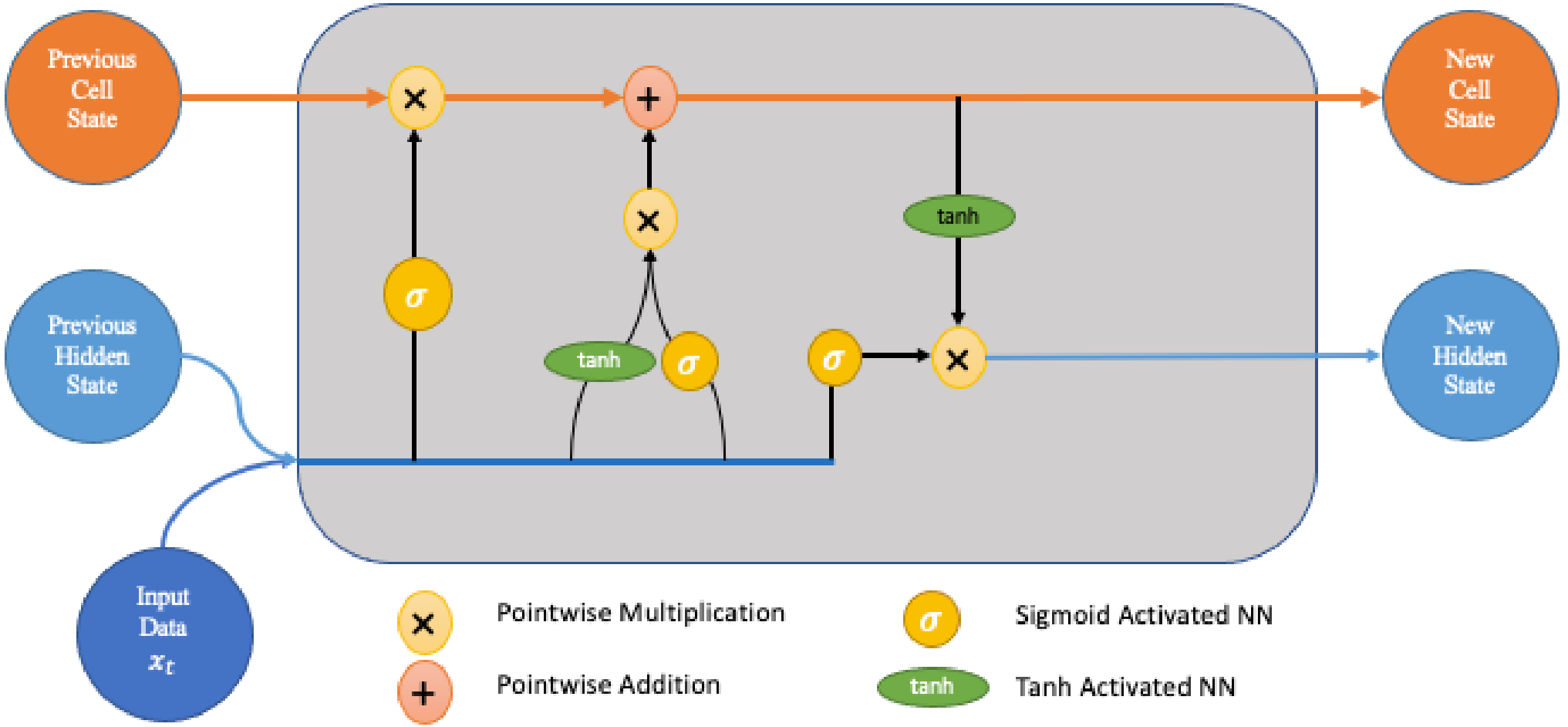
Классификация



LSTM (Long Short Term Memory, 1997)



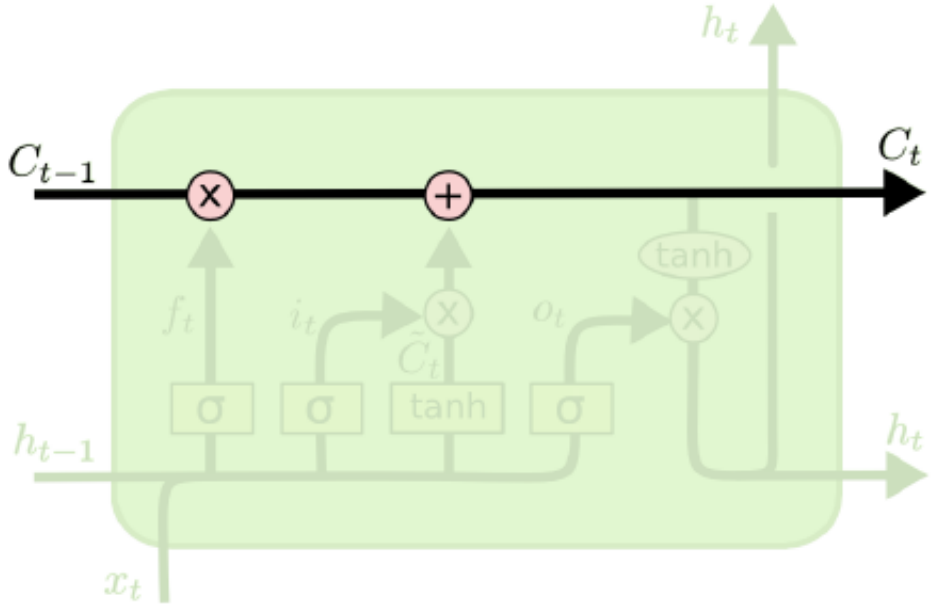
LSTM (Long Short Term Memory)

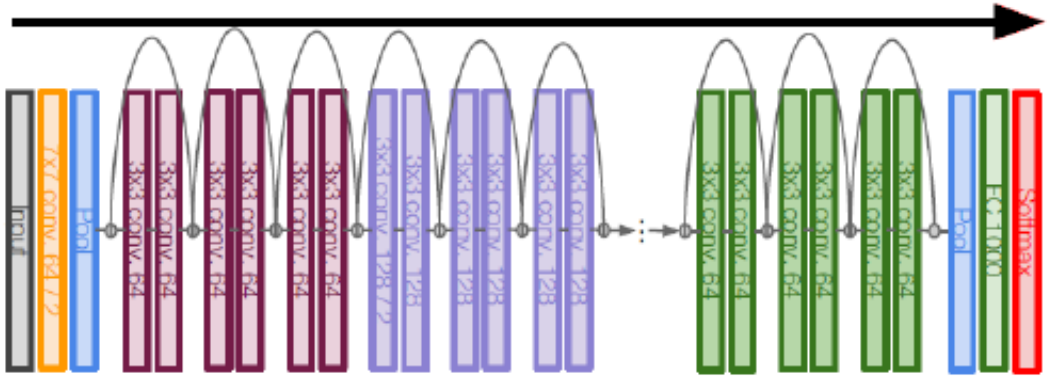


Pointwise = поэлементно
(не «векторо/матрично»)

LSTM (Long Short Term Memory)

«состояние ячейки/блока» – проходит через все блоки
«shortcut connection»

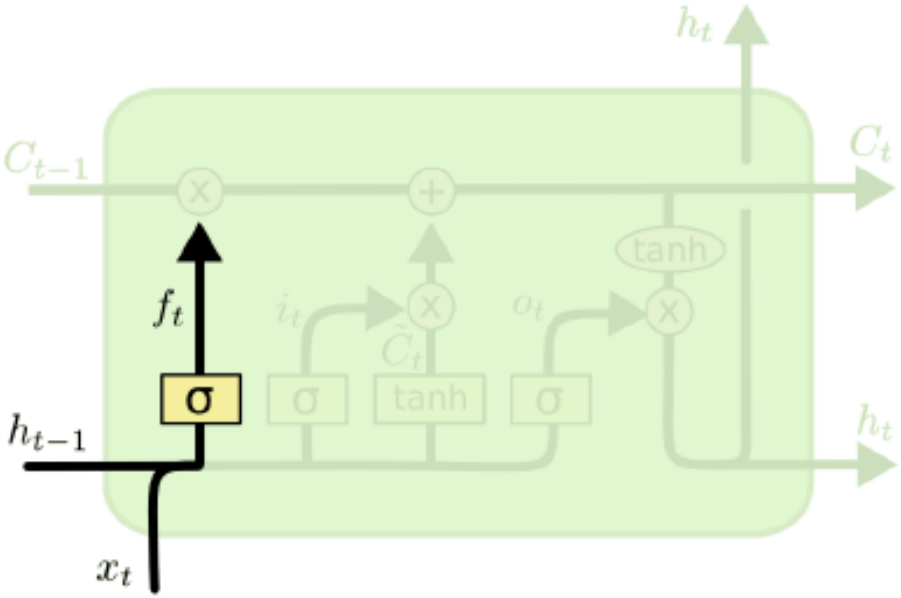




- **память** – перенос информации, которая должна «слабо меняться»
- **борьба с затухающим градиентом** свободно протекает, как в ResNet

LSTM (Long Short Term Memory)

Забывающий гейт (Forget Gate)



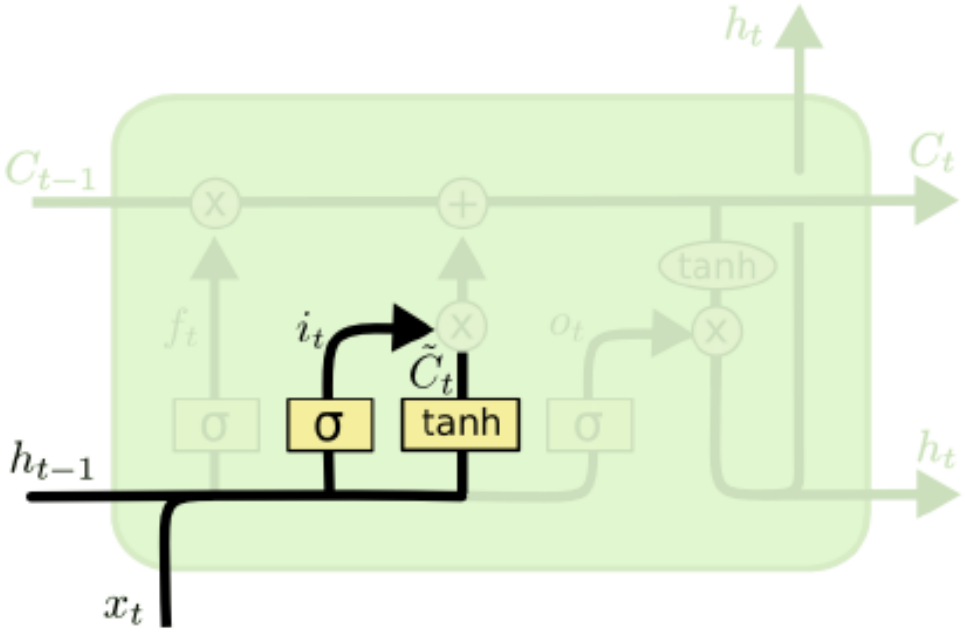
$$f_t = \sigma(W_f[h_{t-1}; x_t] + b_f)$$

если = 1 – передаём полностью состояние блока

если = 0 – то забываем предыдущее состояние

LSTM (Long Short Term Memory)

Входной гейт (Input Gate)



входной гейт:

$$i_t = \sigma(W_i[h_{t-1}; x_t] + b_i)$$

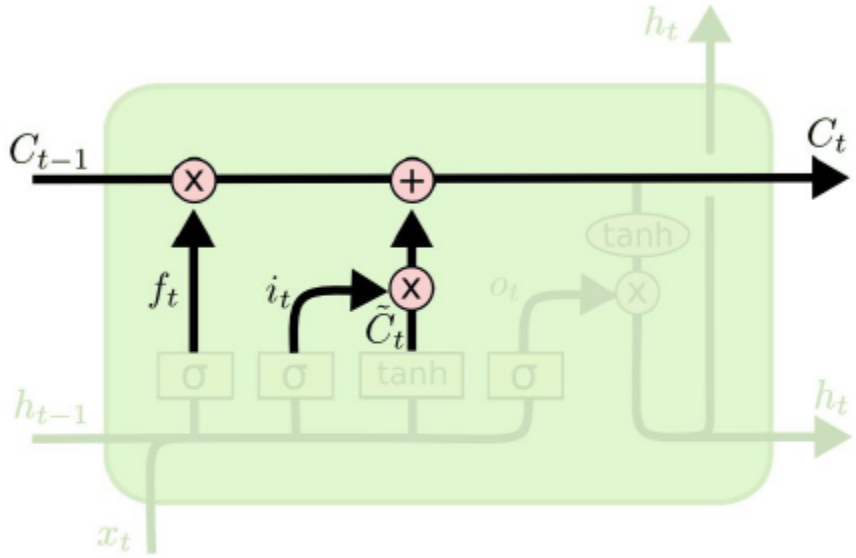
текущее состояние:

$$\tilde{C}_t = \tanh(W_C[h_{t-1}; x_t] + b_C)$$

Какую новую информацию учитываем в состоянии...

LSTM (Long Short Term Memory)

Обновление состояния (Cell update)

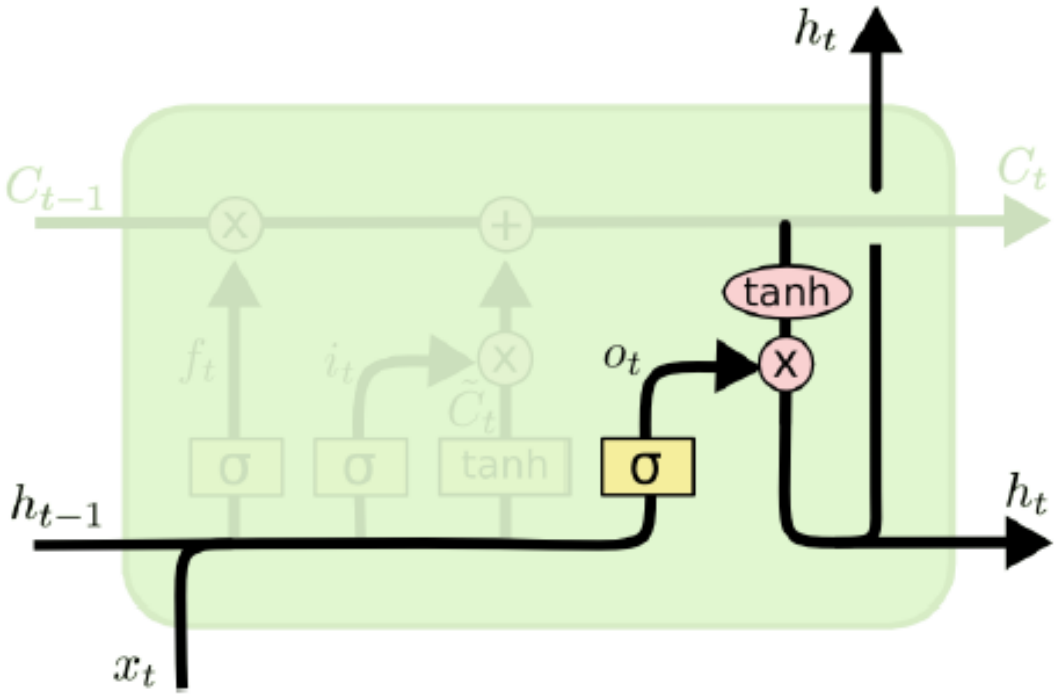


$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$

новое состояние = (старое состояние | гейт) + (посчитанное состояние | гейт)

LSTM (Long Short Term Memory)

Выходной гейт (Output Gate)



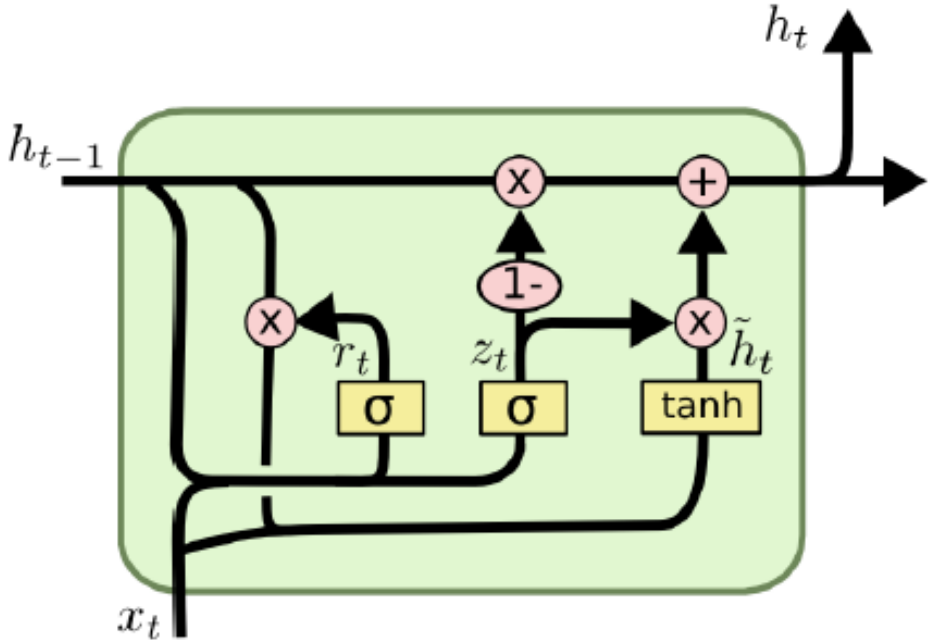
выходной гейт:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

скрытое состояние:

$$h_t = o_t \tanh(C_t)$$

GRU (Gated Recurrent Unit, 2014)



$$z_t = \sigma(W_i[h_{t-1}, x_t] + b_z)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

$$\tilde{h}_t = \tanh(W[r_t h_{t-1}, x_t])$$

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$$

LSTM

```python
class TimeSeriesDataset(Dataset):
    def __init__(self, X, y, sequence_length, forecast_horizon):
        self.X = X
        self.y = y
        self.seq_length = sequence_length
        self.horizon = forecast_horizon

        self.items = []
        full_length = len(X) - sequence_length - forecast_horizon + 1
        for idx in range(full_length):
            x_seq = X[idx:idx + sequence_length]
            y_seq = y[idx + sequence_length:idx + sequence_length + forecast_horizon]

            if np.isnan(x_seq).any() or np.isnan(y_seq).any():
                continue
            self.items.append((x_seq, y_seq))

    def __len__(self):
        return len(self.items)

    def __getitem__(self, idx):
        X_seq, y_seq = self.items[idx]
        return torch.FloatTensor(X_seq), torch.FloatTensor(y_seq)

class LSTMModel(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, bidirectional, output_size, forecast_horizon):
        super(LSTMModel, self).__init__()
        self.hidden_size = hidden_size
        self.num_layers = num_layers
        self.bidirectional = bidirectional
        self.forecast_horizon = forecast_horizon

        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            bidirectional=bidirectional,
            batch_first=True,
            dropout=0.1 if num_layers > 1 else 0
        )
```

LSTM

```
    self.lstm = nn.LSTM(
        input_size=input_size,
        hidden_size=hidden_size,
        num_layers=num_layers,
        bidirectional=bidirectional,
        batch_first=True,
        dropout=0.1 if num_layers > 1 else 0
    )

    lstm_output_size = hidden_size * 2 if bidirectional else hidden_size

    self.regressor = nn.Sequential(
        nn.Linear(lstm_output_size, 32),
        nn.BatchNorm1d(32),
        nn.ReLU(),
        nn.Dropout(0.1),
        nn.Linear(32, output_size * forecast_horizon)
    )

def forward(self, x):
    # x shape: (batch_size, sequence_length, input_size)
    lstm_out, (hidden, cell) = self.lstm(x)
    # Используем только последний выход LSTM
    last_output = lstm_out[:, -1, :]

    output = self.regressor(last_output)
    # Преобразуем в форму (batch_size, forecast_horizon, output_size)
    output = output.view(-1, self.forecast_horizon, 1)

    return output
```

```
'hidden_size': 64, #32
'num_layers': 2,
'bidirectional': True,
'learning_rate': 1e-5,
'sequence_length': 15,
'forecast_horizon': 5,
# 'weight_decay': 0.01,
'grad_clip': 0.01,
```

LSTM

Минутка кода (https://www.kaggle.com/junkoda/pytorch-lstm-with-tensorflow-like-initialization)

```
class Model(nn.Module):
def __init__(self, input_size):
    hidden = [400, 300, 200, 100]
    super().__init__()
    self.lstm1 = nn.LSTM(input_size, hidden[0],
                         batch_first=True,
                         bidirectional=True)
    self.lstm2 = nn.LSTM(2 * hidden[0], hidden[1],
                         batch_first=True,
                         bidirectional=True)
    self.lstm3 = nn.LSTM(2 * hidden[1], hidden[2],
                         batch_first=True,
                         bidirectional=True)
    self.lstm4 = nn.LSTM(2 * hidden[2], hidden[3],
                         batch_first=True,
                         bidirectional=True)
    self.fc1 = nn.Linear(2 * hidden[3], 50)
    self.selu = nn.SELU()
    self.fc2 = nn.Linear(50, 1)
    self._reinitialize()

def forward(self, x):
    x, _ = self.lstm1(x)
    x, _ = self.lstm2(x)
    x, _ = self.lstm3(x)
    x, _ = self.lstm4(x)
    x = self.fc2(self.selu(self.fc1(x)))
    return x

def _reinitialize(self):
    """
    Tensorflow/Keras-like initialization
    """

    for name, p in self.named_parameters():
        if 'lstm' in name:
            if 'weight_ih' in name:
                nn.init.xavier_uniform_(p.data)
            elif 'weight_hh' in name:
                nn.init.orthogonal_(p.data)
            elif 'bias_ih' in name:
                p.data.fill_(0)
                # Set forget-gate bias to 1
                n = p.size(0)
                p.data[(n // 4):(n // 2)].fill_(1)
            elif 'bias_hh' in name:
                p.data.fill_(0)
        elif 'fc' in name:
            if 'weight' in name:
                nn.init.xavier_uniform_(p.data)
            elif 'bias' in name:
                p.data.fill_(0)
```

LSTM: стекаем и распараллеливаем (~2015)

- Nested LSTM
- Grid LSTM

Ковариаты

Wiki: depending on the context, an independent variable is sometimes called a "predictor variable", regressor, covariate, "controlled variable", "manipulated variable", "explanatory variable", exposure variable (see reliability theory), "risk factor" (see medical statistics), "feature" (in machine learning and pattern recognition) or "input variable."

The LSTM model (1995) has been around for some time, whereas TFT (2019) and N-HiTS (2022) are relatively new models. These three neural network models provide the different way the utilize covariates.

LSTM is a recurrent neural network (RNN) and expects covariates that extend into the future until the forecast horizon. Within the Darts framework, those are called *future_covariates*.
N-HiTS use *past_covariates*, known until the point in time when a forecast is generated. Last but not least, the TFT model supports both *past_covariates* and *future_covariates* as well as *static_covariates*.

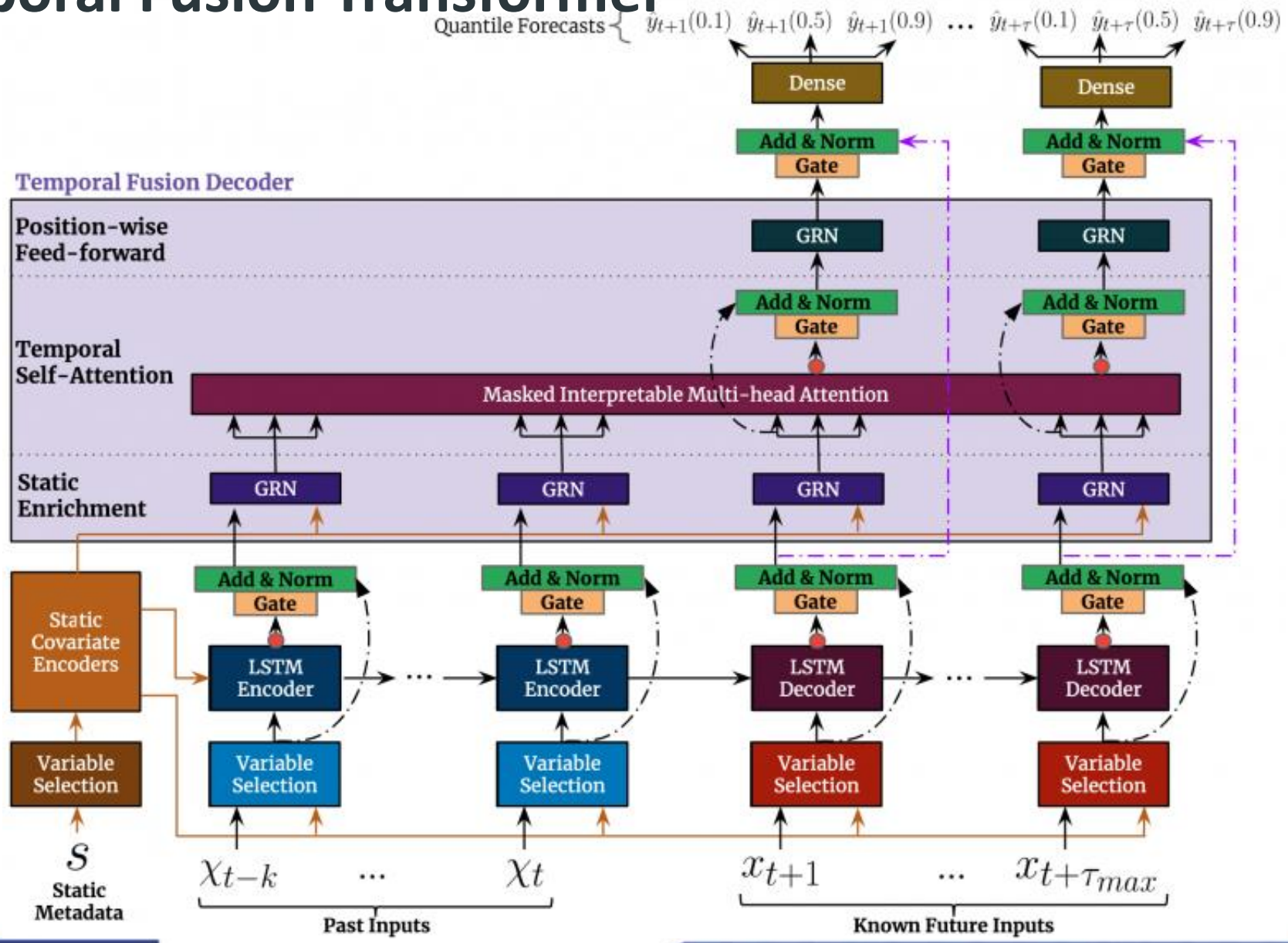
Ковариаты

Ковариаты прошлого – это временные ряды с известными значениями до момента прогнозирования.

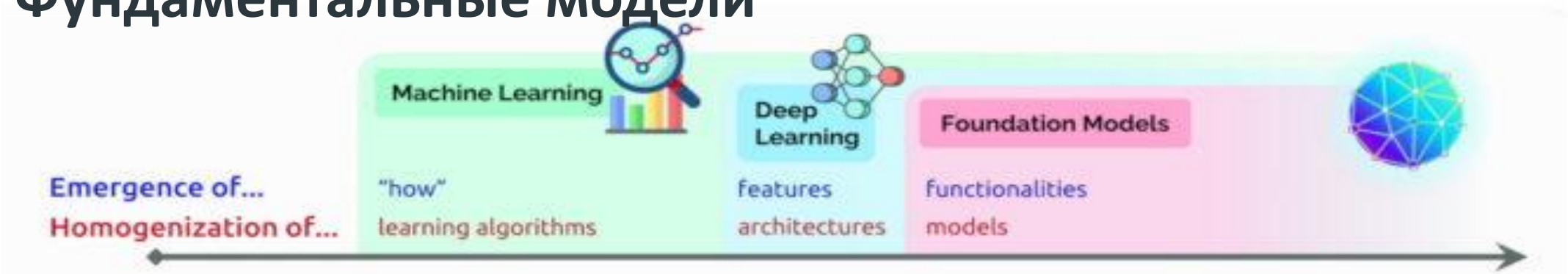
Ковариаты будущего – это временные ряды, значения которых известны не только в прошлом, но и на весь горизонт прогнозирования.

Статические ковариаты – постоянные признаки (ряды констант: как категориальные, так и числовые).

Temporal Fusion Transformer



Фундаментальные модели

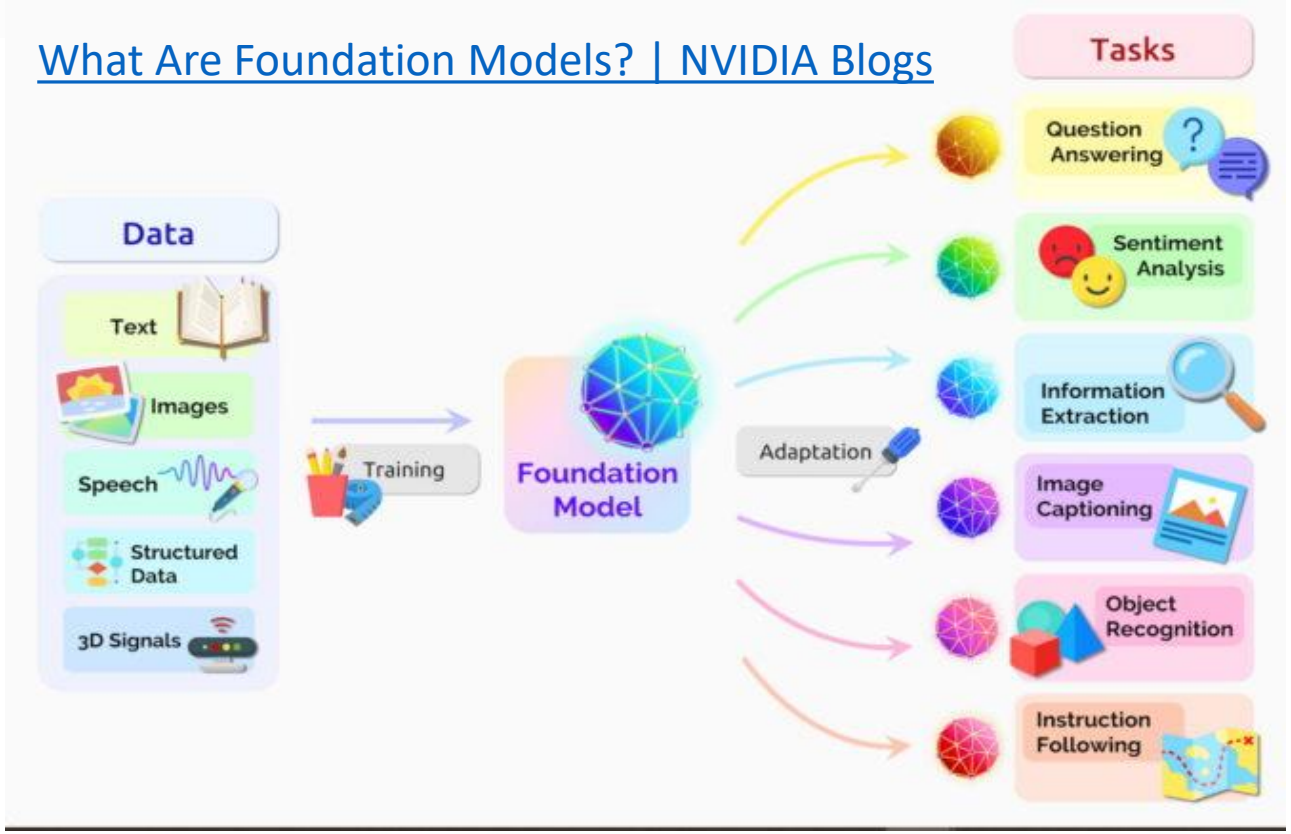


A foundation model is an AI neural network trained on mountains of raw data, generally with unsupervised learning

TimesFM

Hugging Face

[What Are Foundation Models? | NVIDIA Blogs]



Спасибо за внимание!



Запорожцев Иван Федорович
zaporozhtsev.if.work@gmail.com